



LOOP-DEV-101

Build Your Own *Loop* Harness

The model isn't the hard part. The loop around it is.

Sushil Singh · Chakravyuh · Maahaa *dev*

THE PROBLEM

problem LLMs write code that **looks right** and **is wrong**.

the fix has a name a **harness** — the code around the model.

thesis the **harness**, not the model, decides reliability at scale.

VISION · THE BUG YOU'D SHIP

src/app/pricing.ts

```
if (quantity > 10) discount = 0.1;    // should be ≥ 10
```

Buy exactly 10 units → no discount. Looks fine. Passes a glance. Ships.

SINGLE SHOT • NO LOOP

A task without a check is just *hope*.

Ask a model once. Maybe right, maybe wrong. No way to know.

THE WHOLE IDEA, ONE PICTURE



↺ loop until the **gate** passes. The gate is the thing that **can't lie**.

LEVEL 1 · THE RALPH LOOP

One maker. One gate. Retry. Cap at 5.

real Ralph, unabridged (Geoffrey Huntley)

```
while ;; do cat PROMPT.md | claude-code ; done // no stop
```

Adding a cap is where you **start engineering the loop.**

DEMO

```
$ npm run level1
```

← watch the terminal. Say it with me: maker · gate · feedback · retry.

LEVEL 2 · MEMORY – THE ONE CHARACTER

level 1 – overwrite

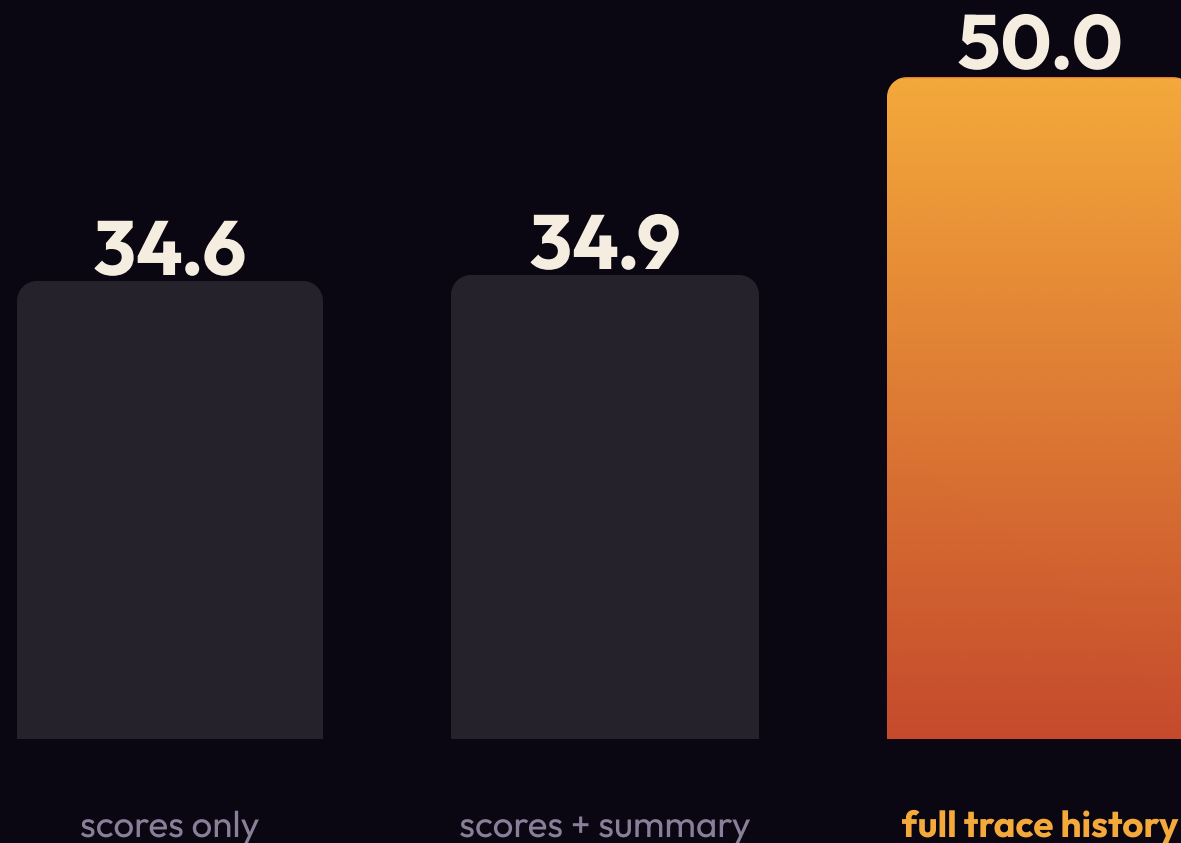
```
feedback = trace;
```

level 2 – accumulate

```
feedback += trace;
```



THE FEEDBACK CHANNEL IS THE LEVER • META-HARNESS (STANFORD, 2026)



Full history vs. compressed signal. That's = vs +=.

DEMO

```
$ npm run level2
```

← same loop. one character different. the maker now remembers.

LEVEL 3 · THE JURY

Gate passes $\neq$ done.
Add an independent vote.

checker — against the spec · **reviewer** — against quality

concurrent · neither sees the other · both must pass

Not a second opinion — an **independent** one.

ROUTE BY ROLE · ONE ENDPOINT, ONE KEY

maker **glm-5.2** — strong

checker **deepseek-v4-flash** — cheap, fast

reviewer **kimi-k2-thinking** — the hard call

War story: the thinking reviewer broke the parser. Fix = 2 lines.

DEMO

```
$ npm run level3
```

← gate passes → two independent judges vote → accepted.

THE PLUMBING · SWAP THE BRAIN

One function. Any OpenAI-shaped endpoint.

Change LLM_BASE_URL → OpenAI · Groq · DeepSeek ·
OpenRouter · local Ollama. No SDK.

LEVEL 4 · NO STARTING CODE — SPEC → TEST → IMPL

test-writer (RED) $\neq$ maker (GREEN)

The maker never writes the test — so it can't grade its own homework. That's what keeps the gate honest.

DEMO

```
$ npm run level4
```

← spec → RED test → maker writes code → GREEN. no bug to seed.

NEWS · THE SAME SHAPE THE 2026 LITERATURE FORMALIZES

E Execution loop — the while-loop, when to stop

V Evaluation interface — the check + its trace

T Tool registry — what the model may call

S State store — memory across attempts

C Context manager — what fits in the prompt

L Lifecycle hooks — run around each step

loop-dev-101 lights up **E V T** → add **pi**, gain **L** → add **chakravyuh**, gain **S C**

SURPRISE • POINT THE LOOP AT ITSELF

The loop is so general you can turn it **on the loop.**

Meta-Harness: same four boxes, writing the harness
instead of the app. +7.7 pts, ~4× fewer tokens.

WHAT YOU CAN DO NOW

- 1 A harness is one shape: **maker** → **gate** → **feedback** → **retry**.
- 2 The gate is the thing that can't lie. A task without a check is just hope.
- 3 The lever is the **feedback channel**, not the loop. $34.6 \rightarrow 50.0 = = \rightarrow +=$.
- 4 **Independence**, not repetition, is what a second judge buys you.
- 5 The same four boxes scale from 250 lines to production to the research frontier.



Maahaa *dev*

The model isn't the hard part.

The loop around it is —
and the loop is just a **task**
with a check.

Clone `loop-dev-101`. Point the loop at something that matters to you.